

NLP

Modelos de lenguaje con redes LSTM

Docente:
Josselyn Ordóñez

Redes Neuronales Recurrentes (RNNs)



Es un tipo de neurona con un estado interno (o memoria) de manera que la información del pasado influye en los resultados futuros.



Se utiliza principalmente para resolver problemas de secuencia, en donde el valor anterior está relacionado con el valor futuro.



Permite construir modelos cuyos tamaños de secuencia sean potencialmente arbitrarios.



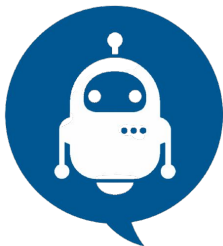
Implementa modelos de lenguaje de la forma:

$$\prod_{i=1}^{i=m} P(w_i | w_{i-(n-1)}, \dots, w_{i-1})$$

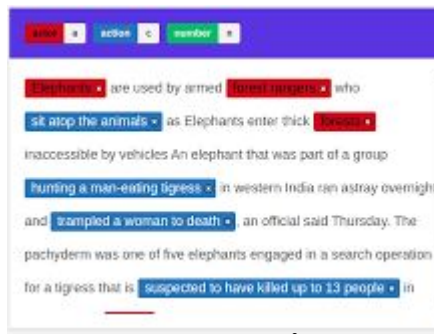
Cual es la probabilidad de que aparezca
cierta palabra en base a las palabras pasadas

"Hoy el **día** está **hermoso** y **despejado**, se puede ver un hermoso **cielo... azul**"

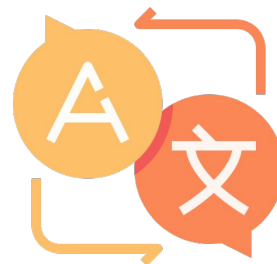
Algunos problemas de secuencia



Bots
Conversacionales



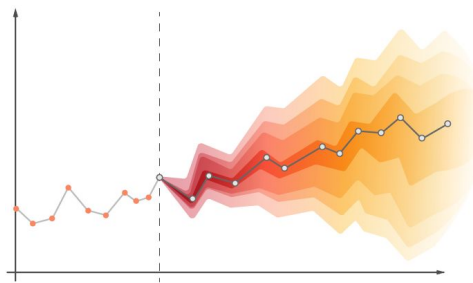
Name entity
recognition



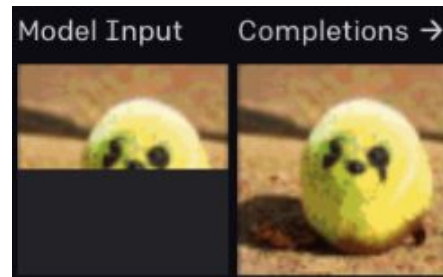
Traducción de
idiomas



Speech to text



Series
temporales



Completar una
imagen

Celda RNN básica (Elman)

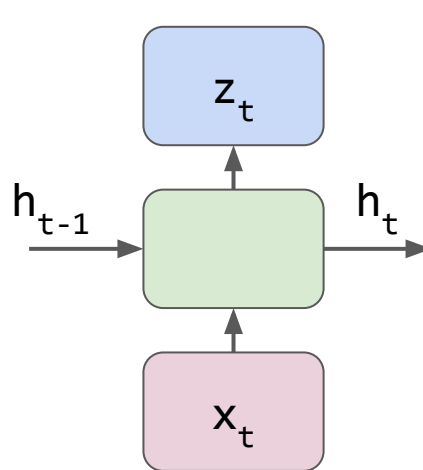
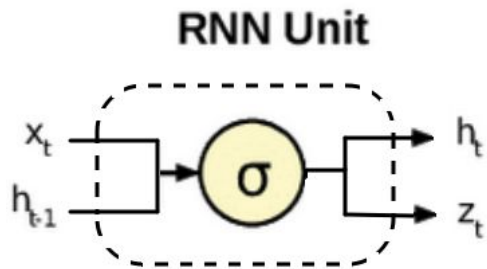
[LINK](#)

[API KERAS](#)

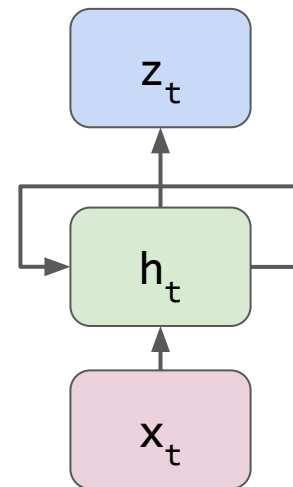


En Keras: SimpleRNN

$$h_t = \sigma(W_{hh} * h_{t-1} + W_{hx} * x + b_h)$$
$$z_t = h_t$$



Unidad básica



Representación compacta

Esta arquitectura introduce una dimension (del tiempo) adicional a la profundidad de la red neuronal. Ahora existe una secuencia de tiempo/orden, lo que afecta al backpropagation. Al ser mas profundo la red, se genera el problema de Vanishing Gradient. Una forma de solucionar esto es con LSTM

Long short term memory (LSTM)

[LINK](#)



Se introduce este tipo de celda neuronal con mayor persistencia de memoria para lograr capturar relaciones de palabras a largo plazo.



Se crearon en 1997. Se adoptó como la layer principal para problemas de secuencia en 2014 hasta la aparición de los transformers en 2017.



Desplazaron completamente a las capas RNN simples (Elman), ya que el costo adicional de las LSTM es marginal respecto al beneficio que otorgan



Se basan en el principio de ponderar la importancia de una palabra respecto al contexto futuro/pasado (key words).

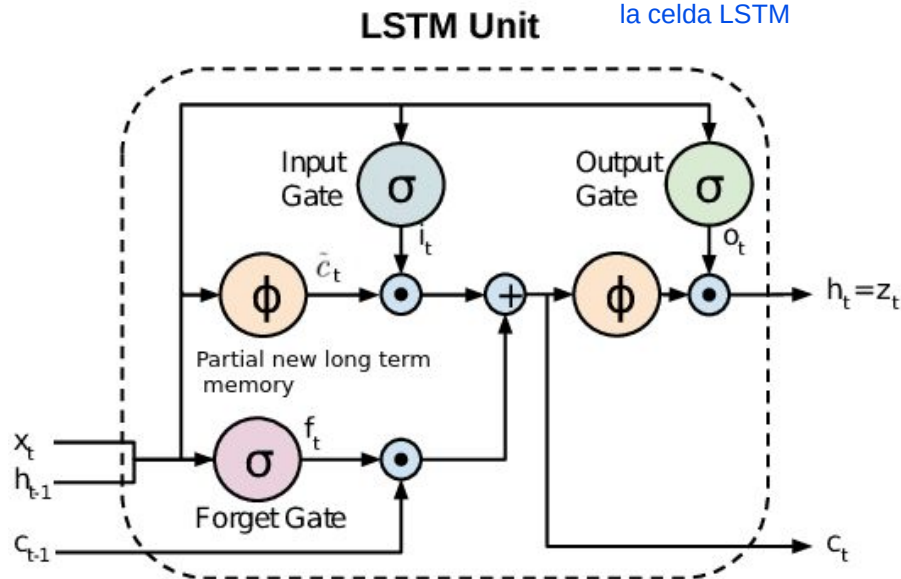
¿Comprarías este producto?

*"**Incredible!** El producto es lo que venden, hace lo que tiene que hacer y me **ayudó mucho** a resolver los problemas que tenía. Lo **volvería a comprar** sin dudas"*

LSTM approach - Idea de la arquitectura



En cierta forma existen 4 unidades que conforman la celda LSTM



Input (i): Dado el último estado (h_{t-1}) evalúa cuánto de la nueva entrada (x) se incluirá en la memoria de largo plazo (C_t).

Forget (f): Dada la entrada (X) cuánto del estado de memoria anterior (C_{t-1}) tiene importancia en el nuevo estado.

Output (o): Qué parte del estado de memoria (C_t) pasa al próximo estado de memoria h_t .

$\cdot$: Producto elemento a elemento

ϕ : Función Tanh

$+$: Suma vectorial

$\rightarrow$: Conexiones

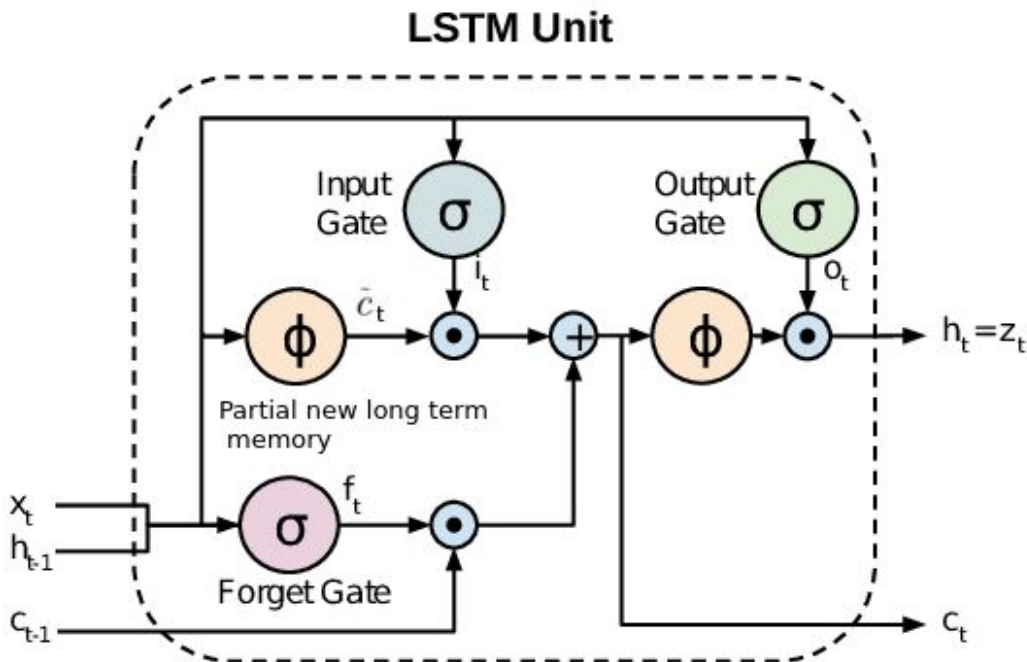
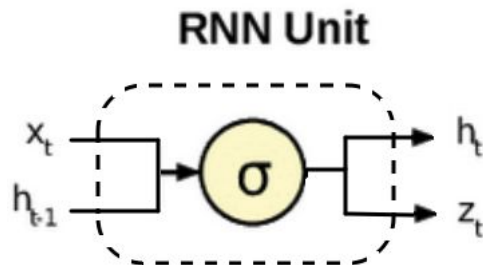
C: 'Memoria a largo plazo'

Como no pasa por una función de activación, pasa más directo de capa en capa. No se ve afectado por el backpropagation

LSTM vs RNN



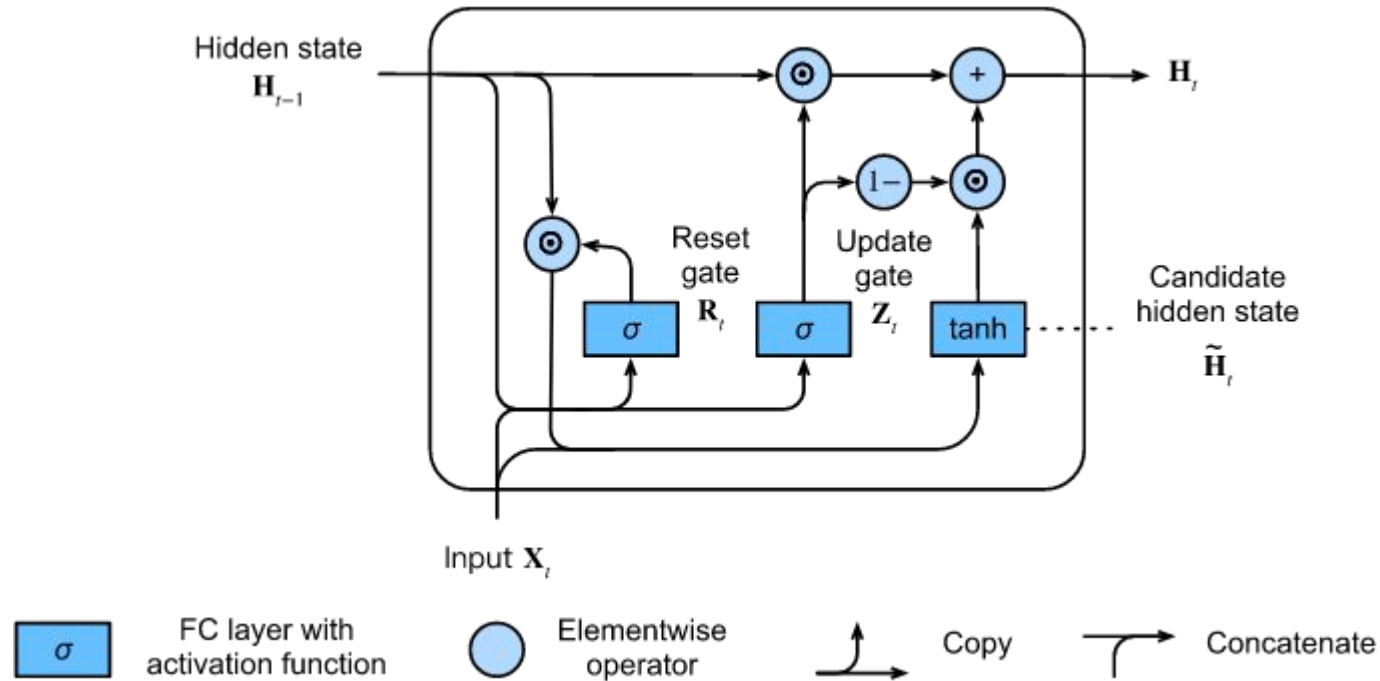
La memoria de largo plazo c_t permite propagar gradiente eficientemente a mayor “profundidad temporal”.



Gated Recurrent Units (GRU)



Esta red es un poco mas eficiente que LSTM. Aplica el concepto de largo plazo al reutilizar el estado interno H como memoria



RNN, LSTM y GRU son arquitecturas secuenciales. Se utilizan, generalmente, para prediccion/generacion de texto

Predicción de texto - Modelos de lenguaje



Objetivo: Entender la estructura estadística del lenguaje aprovechando su estructura secuencial.

Tarea: Aprender a predecir la siguiente palabra.

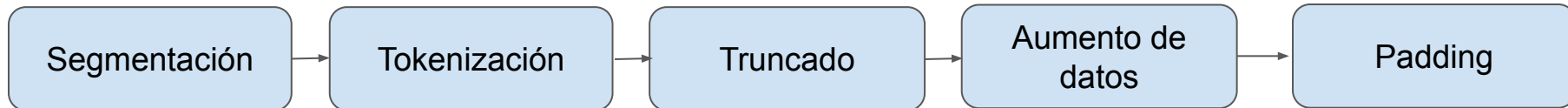
$$\prod_{i=1}^{i=m} P(w_i | w_{i-(n-1)}, \dots, w_{i-1})$$

The next word

$$(X_1, \dots, X_n) \rightarrow X_{n+1}$$

Pasos a seguir:

Identificación de las partes del texto



Dividir el texto en partes mas chicas para procesar el contenido semantico de cada parte

Segmentación y Tokenización



Documentos:

En este ejemplo, la segmentación
es por palabra separados por el espacio.
No se incluye los signos de puntuación

'Yesterday, all my troubles seemed so far away'

Segmentación:

['yesterday', 'all', 'my', 'troubles', 'seemed', 'so', 'far', 'away']

Tokenización:

→ [34, 189, 200, 1345, 8, 69, 12,753]

Ventana de contexto



La ventana de contexto en un modelo de lenguaje es la cantidad máxima de tokens (palabras o subpalabras) que el modelo puede considerar a la vez como entrada. Define el alcance del "pasado" que el modelo puede usar para predecir la siguiente palabra o generar texto. [En general, este parametro se define en el entrenamiento](#)

Si la secuencia es más **CORTA**
que la ventana de contexto



Padding (relleno)

Si la secuencia es más **LARGA**
que la ventana de contexto



Truncado

Truncado de secuencias:



Si tenemos una secuencia de tamaño M y una ventana de contexto de tamaño N tal que $M > N$, entonces podemos generar $1 + M - N$ secuencias de tamaño N :

Ejemplo ($M=6$, $N=3$)

["Todas", "las", "hojas", "son", "del", "viento"] se convierte en:

```
["Todas", "las", "hojas"]  
["las", "hojas", "son"]  
["hojas", "son", "del"]  
["son", "del", "viento"]
```

Data Augmentation y padding



Queremos que el modelo sea capaz de predecir cada elemento de la secuencia, no solo el último:

`["Todas", "las", "hojas", "son", "del", "viento"]` $\longrightarrow$ `[1, 2, 3, 4, 5, 6]`

Ejemplo (M= 6, N=3)

Con el truncado se generaron
4 secuencias, pero con el padding
se generan aun mas.

`["Todas", "las", "hojas"]`
`["las", "hojas", "son"]`
`["hojas", "son", "del"]`
`["son", "del", "viento"]`

`["Todas"]` $\rightarrow$ `[0, 0, 1]`
`["Todas", "las"]` $\rightarrow$ `[0, 1, 2]`
`["Todas", "las", "hojas"]` $\rightarrow$ `[1, 2, 3]`

Este Data Augmentation se hace porque el objetivo es predecir texto.
Y en general se empieza de menos palabras a mas, ayuda al modelo
a generar mejor.

La biblioteca de Babel



“No hay en la vasta Biblioteca, dos libros idénticos. De esas premisas incontrovertibles dedujo que la Biblioteca es total y que sus anaqueles registran todas las posibles combinaciones de los veintitantos símbolos ortográficos (número, aunque vastísimo, no infinito) o sea todo lo que es dable expresar: en todos los idiomas. Todo[...]”



$$\begin{aligned} P_{LM} (x^{T+1}, x^T, \dots, x^1) &= P_{LM} (x^{T+1} | x^T, \dots, x^1) P_{LM} (x^T, x^{T-1}, \dots, x^1) \\ &= P_{LM} (x^{T+1} | x^T, \dots, x^1) P_{LM} (x^T | x^{T-1}, \dots, x^1) P_{LM} (x^{T-1}, \dots, x^1) \\ &= \prod_{t=1}^T P_{LM} (x^{t+1} | x^t, \dots, x^1) \end{aligned}$$

Ejemplo:

$$P(\text{"La hojarasca crepitar"}) = P(\text{"crepitar"} | \text{"La hojarasca"}) P(\text{"hojarasca"} | \text{"la"}) P(\text{"la"})$$

La probabilidad depende de las palabras anteriores, por eso se genera texto en base a las anteriores.



Perplejidad: “Cantidad media de palabras posibles, en promedio.”

Metrica que se utiliza para saber que tan bien son las predicciones

$$\text{perplexity} = \prod_{t=1}^T \left(\frac{1}{P_{\text{LM}}(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)})} \right)^{1/T}$$

Normalized by
number of words
(vocabulario)

Inverse probability of corpus, according to Language Model

A mayor perplejidad, mas confundido esta el modelo (osea es peor). A menor valor, mas seguro esta de lo generado

Esto ayuda que la perplejidad no se muy grande para secuencias largas

The quick brown fox jumps
over the lazy dog

```
ppl( sequence ) = torch.mean(  
[ 89 , 77 , 44 , 12 , 3 , 2 , 28 , 9 ] )
```

```
ppl( sequence ) = 33
```

Guarda! Es el promedio geométrico en verdad: **16,49...**

Midiendo desempeño en modelos de lenguaje: perplexity



$$\text{perplexity} = \prod_{t=1}^T \left(\frac{1}{P_{\text{LM}}(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)})} \right)^{1/T}$$

Normalized by number of words

Inverse probability of corpus, according to Language Model

Por cuestiones de estabilidad numérica conviene operar sobre los logaritmos de las probabilidades.

$$\text{PPL}(X) = \exp \left\{ -\frac{1}{t} \sum_i^t \log p_{\theta}(x_i | x_{<i}) \right\}$$

Cross-entropy:

$$L = -\frac{1}{m} \sum_{i=1}^m y_i \cdot \log(\hat{y}_i)$$

Modelo equiprobable: perplejidad V

Modelo cuya perplejidad es igual al tamaño del vocabulario (V).

Haciendo la exponencial de la Cross-entropy se llega a la PPL. Esto suele ser mas simple de computar que la PPL en sí.



Colab

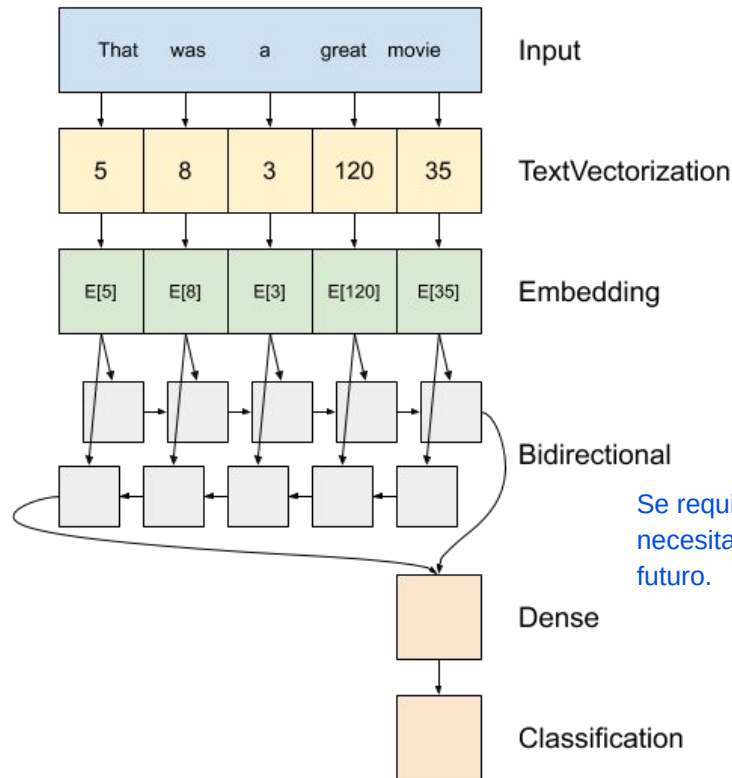


Embeddings + LSTM + Classifier

[LINK](#)



Arquitectura de alto nivel de un modelo “sentiment analysis”



Se requiere una red Bidireccional porque no solo necesita cosas del pasado, si no tambien del futuro.

Generación de texto en modelos de lenguaje:

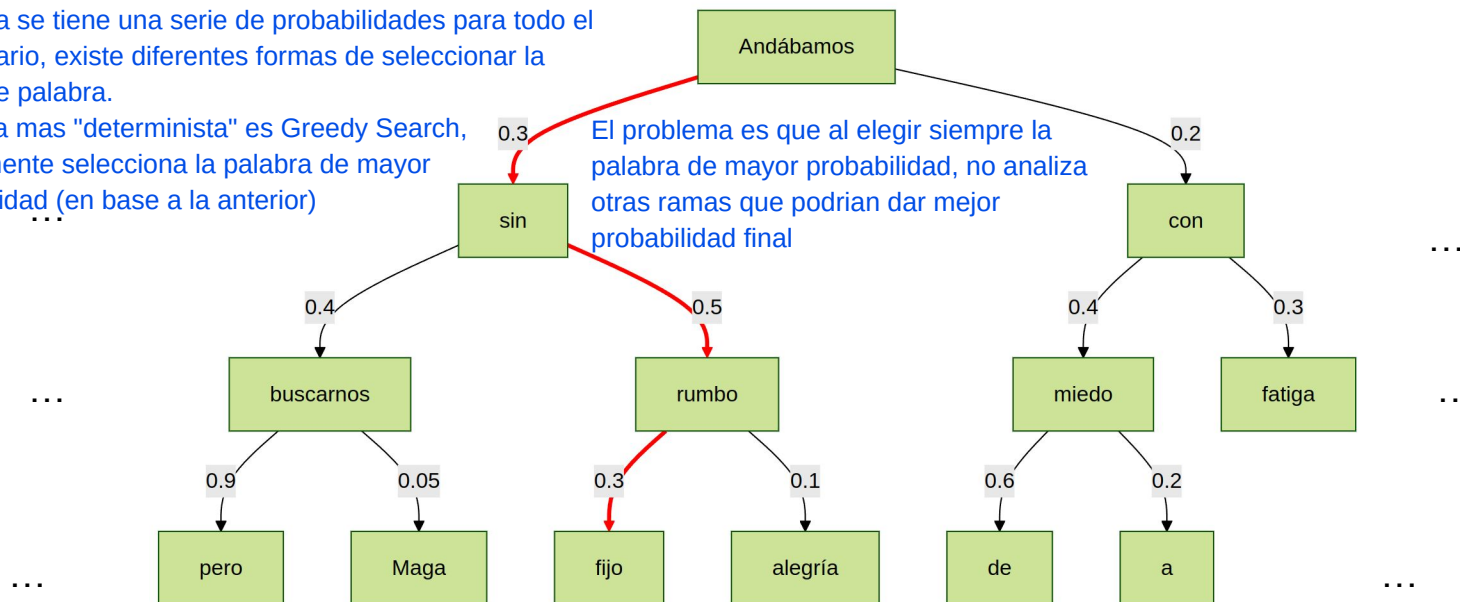
Greedy search



Como ya se tiene una serie de probabilidades para todo el vocabulario, existe diferentes formas de seleccionar la siguiente palabra.

La forma mas "determinista" es Greedy Search, basicamente selecciona la palabra de mayor probabilidad (en base a la anterior)

El problema es que al elegir siempre la palabra de mayor probabilidad, no analiza otras ramas que podrian dar mejor probabilidad final



$$\prod_{t=1}^T P_{LM}(x^{t+1} | x^t, \dots, x^1) \longrightarrow$$

$$P(\text{"Andábamos sin rumbo fijo"}) =$$

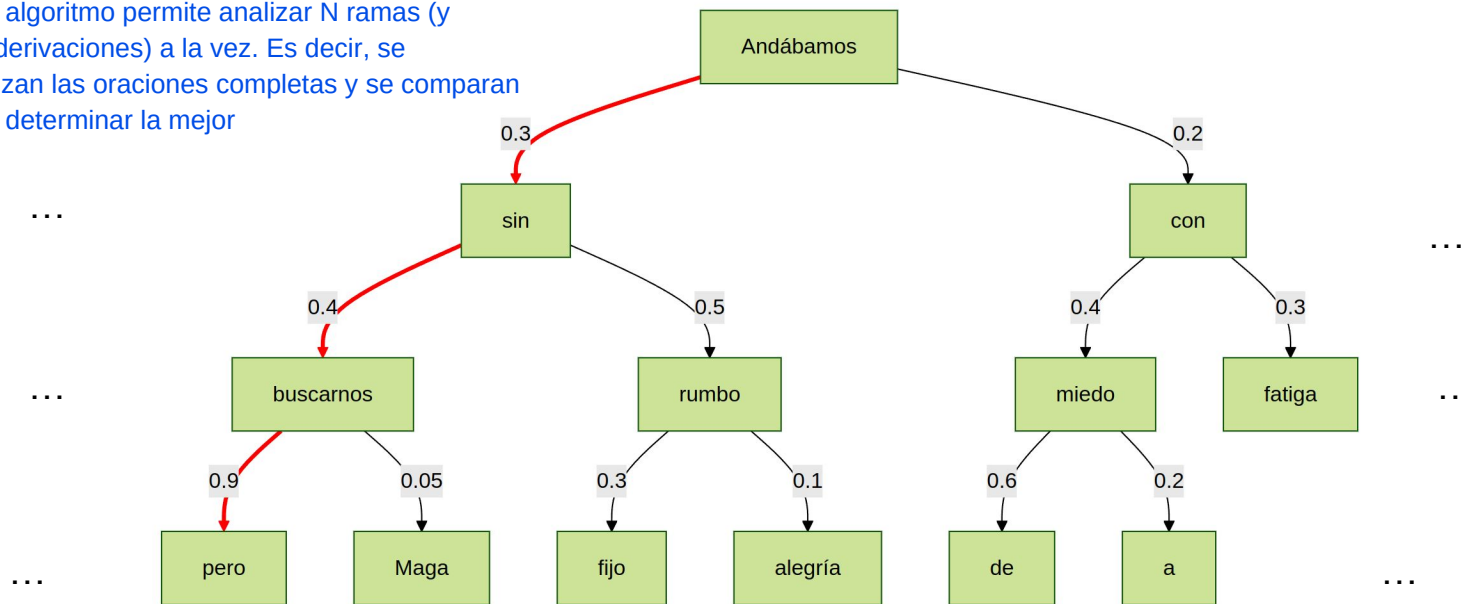
$$0.3 \times 0.045 \times 0.3 = \mathbf{0.045}$$

Generación de texto en modelos de lenguaje:

Beam search



Este algoritmo permite analizar N ramas (y sus derivaciones) a la vez. Es decir, se analizan las oraciones completas y se comparan para determinar la mejor



$$\prod_{t=1}^T P_{LM}(x^{t+1} | x^t, \dots, x^1) \longrightarrow$$

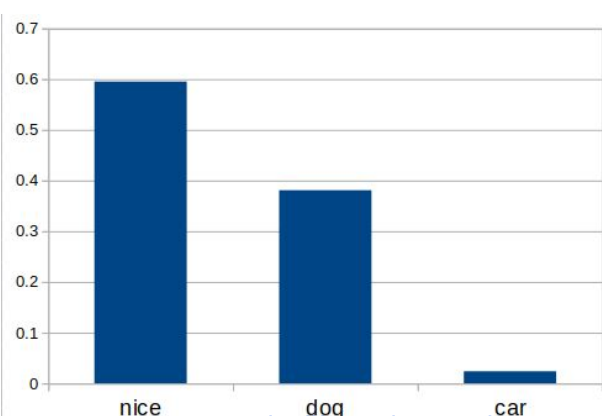
$$P(\text{"Andábamos sin buscarnos, pero"}) = 0.3 \times 0.4 \times 0.9 = \mathbf{0.108}$$

Generación de texto en modelos de lenguaje: Muestreo aleatorio con temperatura



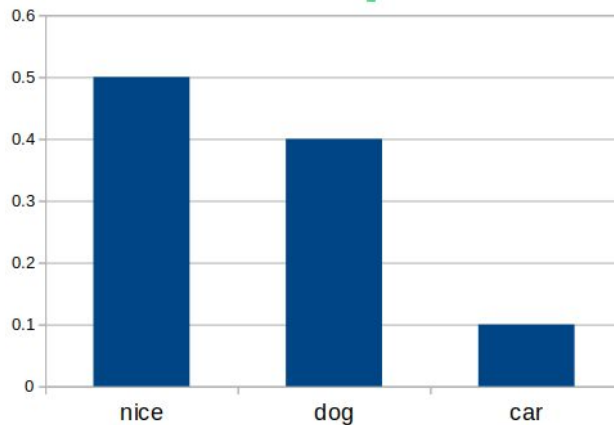
$$P_i = \frac{e^{\frac{y_i}{T}}}{\sum_{k=1}^n e^{\frac{y_k}{T}}}$$

Esta formula es la Softmax, a la cual se le agrega el termino de Temperatura.
Con la temperatura se modifica las probabilidades de cada palabra



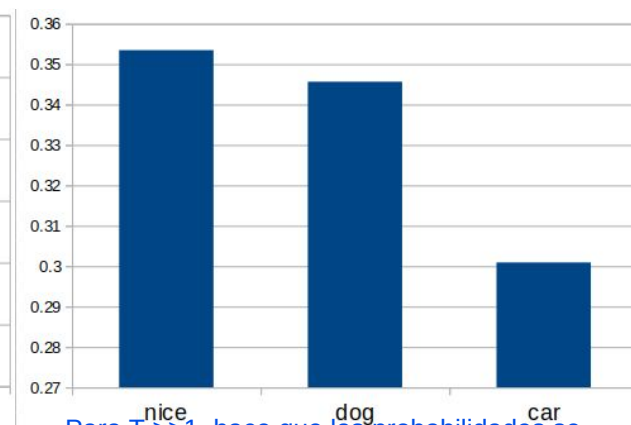
Para $T < 1$, mayor es la diferencia entre las probabilidades de las palabras. Es decir, el modelo responde con mayor determinismo.

Temperatura = 0.5



Se aplica la softmax sin alteracion

Temperatura = 1



Para $T \gg 1$, hace que las probabilidades se parezcan mas. El modelo puede ser mas "creativo"

Temperatura = 10

Pre-trained Embedding layer

[LINK](#)

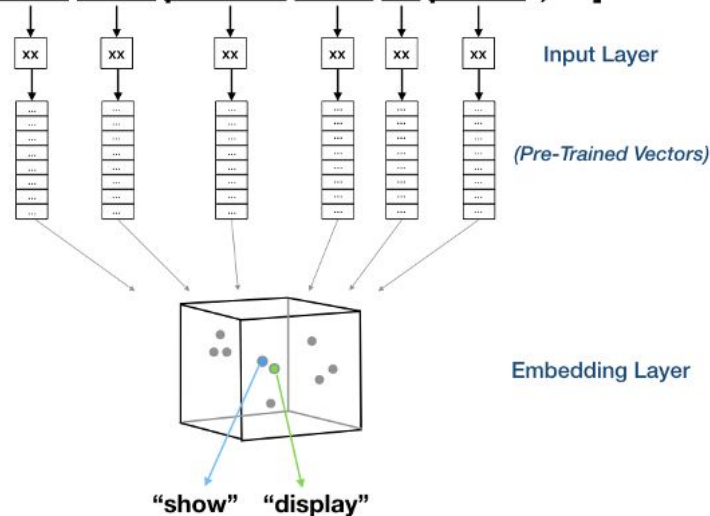
[LINK](#)



"Utilizar embeddings pre-entrenados (GloVe / FastText) en la layer de Embeddings de Keras"

```
Embedding(input_dim=vocab_size, # definido en el Tokenizador
          output_dim=embed_dim, # dimensión de los embeddings utilizados
          input_length=in_shape, # máxima sentencia de entrada
          weights=[embedding_matrix], # matrix de embeddings
          trainable=False)) # marcar como layer no entrenable
```

**[“I want to search for blood pressure result history”,
“Show blood pressure result for patient”, ...]**



a	1
am	2
as	3
act	4
all	5
...	6
...	7
...	8
...	9
...	10
...	11
...	12
...	...
...	100
...	...
...	1000
...	...
...	10000
...	...
...	...



Utilizar otro dataset y
poner en práctica
la generación de
secuencias con las
estrategias presentadas.

